

Connecting Slack to n8n

A step-by-step, illustrated setup guide — receive messages from Slack and send messages to Slack through n8n.

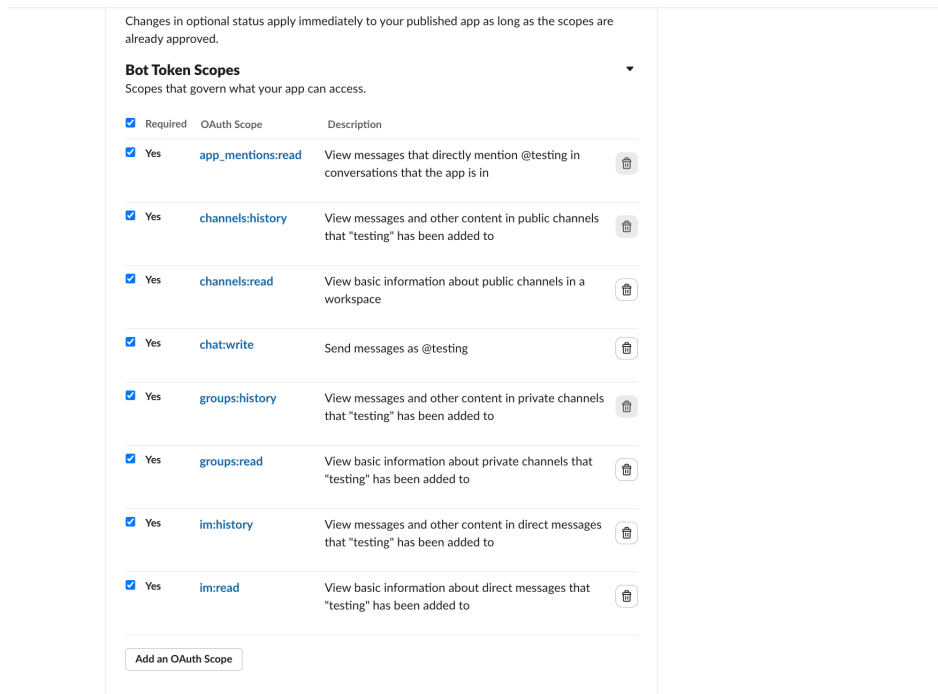
1. Create a Slack App

Go to api.slack.com/apps → **Create New App** → **From scratch**. Give it a name (e.g. "n8n Integration") and select the workspace you want to connect.

2. Add Bot Token Scopes

In your app, go to **OAuth & Permissions** → **Scopes** → **Bot Token Scopes**. Add the following scopes — these control what the bot is allowed to read and send:

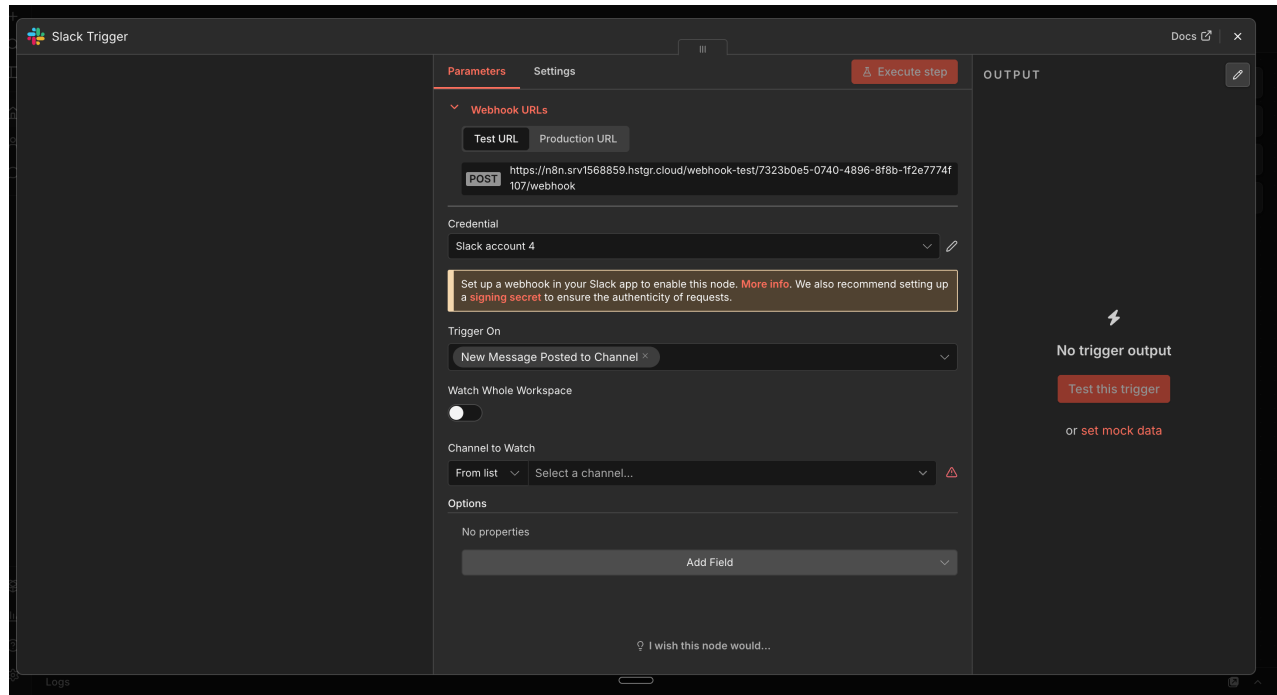
- **chat:write** — send messages as the bot (required for *sending*)
- **channels:read** — list public channels (needed for n8n's channel picker)
- **channels:history** — read messages in public channels (needed for *receiving*)
- **groups:read** — list private channels
- **groups:history** — read messages in private channels
- **im:read** and **im:history** — read direct messages
- **app_mentions:read** — see @mentions of the bot



The full scope list, as it should appear once everything is added. Note both a *:read* and a *:history* scope exist for each channel type — *:read* lets n8n list the channels in its picker, *:history* lets it actually read the messages inside them.

3. Get Your n8n Webhook URL First

In your n8n workflow, add a **Slack Trigger** node. Under **Webhook URLs**, copy the **Test URL** (or Production URL once live) — this is what Slack will send events to. You'll paste it into Slack in the next step.



The Slack Trigger node in n8n. The webhook URL under "Test URL" gets pasted into Slack's Event Subscriptions Request URL field. Note the warning triangle here — that's the channel-picker issue covered in the Troubleshooting table at the end of this guide.

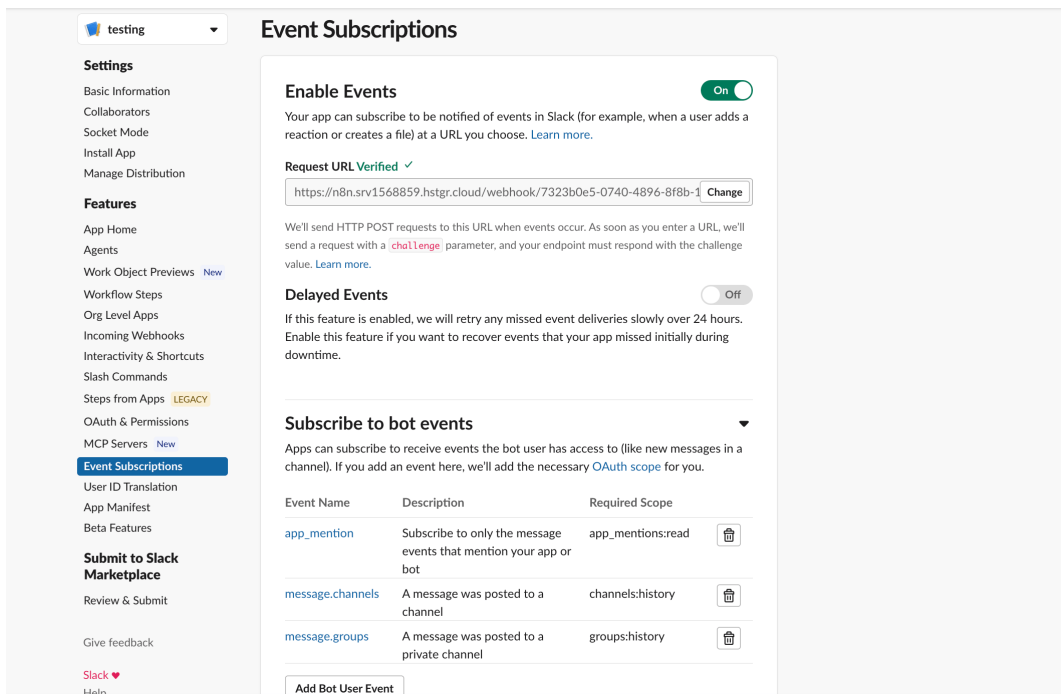
4. Enable Event Subscriptions in Slack

In your Slack app, go to **Event Subscriptions** and toggle **Enable Events** ON. Paste the n8n webhook URL from Step 3 into **Request URL** — Slack sends a verification challenge, and n8n responds to it automatically. You should see a green **"Verified"** checkmark next to Request URL, as shown below.

Under **Subscribe to bot events**, add the events you need to receive:

- **app_mention** — requires the **app_mentions:read** scope
- **message.channels** — a message was posted to a public channel; requires **channels:history**
- **message.groups** — a message was posted to a private channel; requires **groups:history**

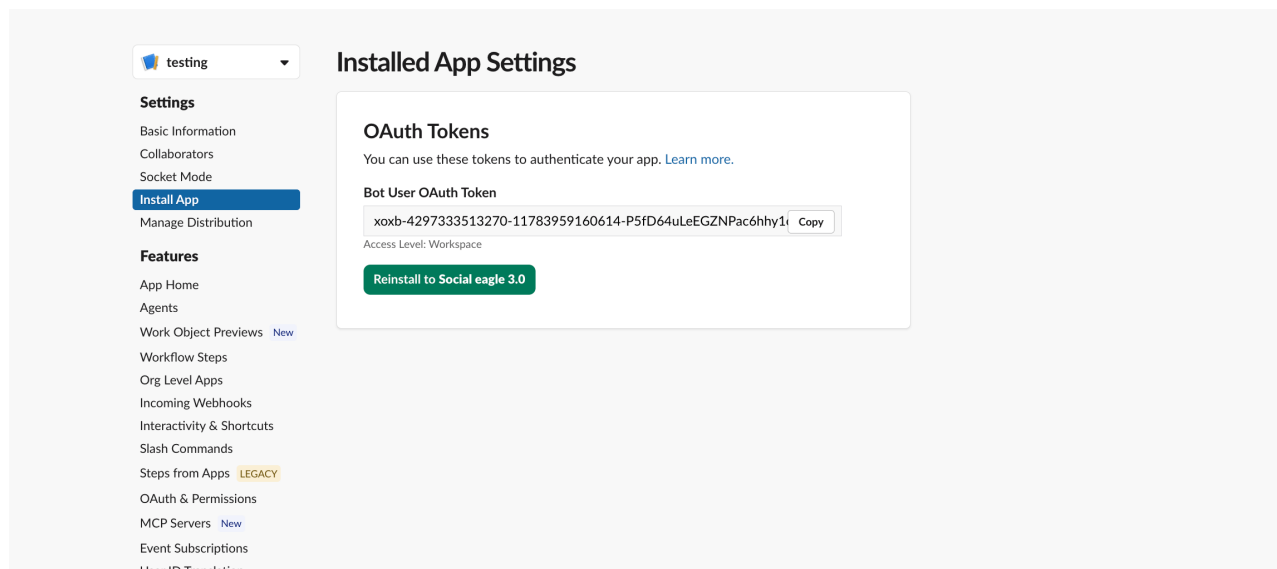
Slack automatically adds the required scope for you when you add an event here — but it's worth double-checking against the list in Step 2.



Event Subscriptions page: Request URL shows "Verified" ✓, and three bot events are subscribed — `app_mention`, `message.channels`, and `message.groups` — each listing its required scope on the right.

5. Install (or Reinstall) the App to Your Workspace

Go to **Install App** in the sidebar. If this is your first install, click **Install to Workspace**. If you changed scopes or events *after* installing, you'll instead see a **Reinstall to [Workspace Name]** button — click that instead. Scopes and events only take effect after an install or reinstall.



Install App page, after install. The **Bot User OAuth Token** (starting with `xoxb-`) is shown here — copy it. This is the value that goes into n8n's **Access Token** field (Step 6). The green **"Reinstall to [Workspace]"** button appears whenever scopes/events changed since the last install — click it to apply your changes.

Your app's **Signing Secret** is a separate value, found under **Basic Information** → **App Credentials**. Click **Show** next to Signing Secret to reveal and copy it.

Settings

- Basic Information
- Collaborators
- Socket Mode
- Install App
- Manage Distribution

Features

- App Home
- Agents
- Work Object Previews New
- Workflow Steps
- Org Level Apps
- Incoming Webhooks
- Interactivity & Shortcuts
- Slash Commands
- Steps from Apps LEGACY
- OAuth & Permissions
- MCP Servers New
- Event Subscriptions
- User ID Translation
- App Manifest
- Beta Features

Submit to Slack Marketplace

- Review & Submit

Give feedback

Slack ♥

Help

Contact

Policies

App Credentials

These credentials allow your app to access the Slack API. They are secret. Please don't share your app credentials with anyone, include them in public code repositories, or store them in insecure ways.

App ID **Date of App Creation**

A0BP3T1F4KU August 9, 2026

Client ID

4297333513270.11785919514674

Client Secret

..... **Show** **Regenerate**

You'll need to send this secret along with your client ID when making your [oauth.v2.access](#) request.

Signing Secret

..... **Show** **Regenerate**

Slack signs the requests we send you using this secret. Confirm that each request comes from Slack by verifying its unique signature.

Verification Token

Olc0Ajlrxld2hBBs7FekVUsy7 **Regenerate**

This deprecated Verification Token can still be used to verify that requests come from Slack, but we strongly recommend using the above, more secure, signing secret instead.

App-Level Tokens

App-level tokens allow your app to use platform features that apply to multiple (or all) installations—for example, the [API to list event authorizations](#). Features have distinct scopes, so request only the scopes for the features you need. Each app can have a maximum of 10 app-level tokens at one time.

Discard Changes **Save Changes**

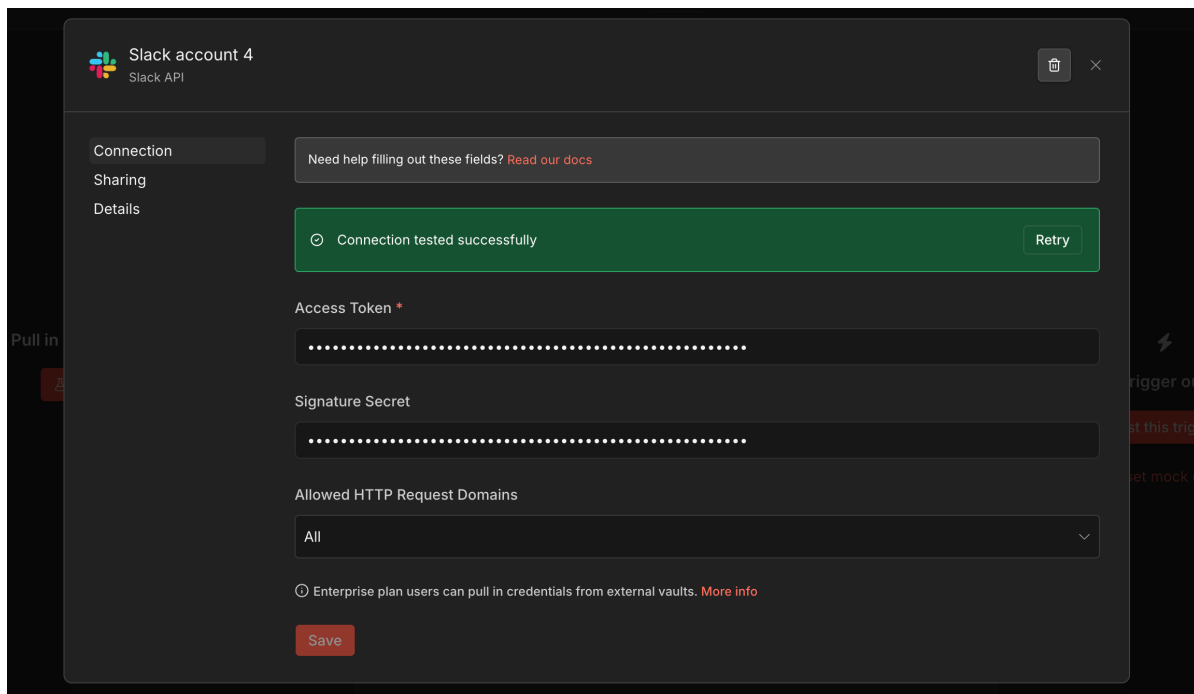
App Credentials page. The **Signing Secret** (middle of the page) is the value that goes into n8n's **Signature Secret** field (Step 6) — it lets n8n verify that incoming requests genuinely came from Slack.

6. Connect the Credential in n8n

In n8n, open (or create) your Slack credential — from the Slack Trigger node's **Credential** dropdown, click the pencil icon to edit, or go to Credentials → New → Slack API.

Bot User OAuth Token (from Slack's Install App page, Step 5) → paste into n8n's **Access Token** field.

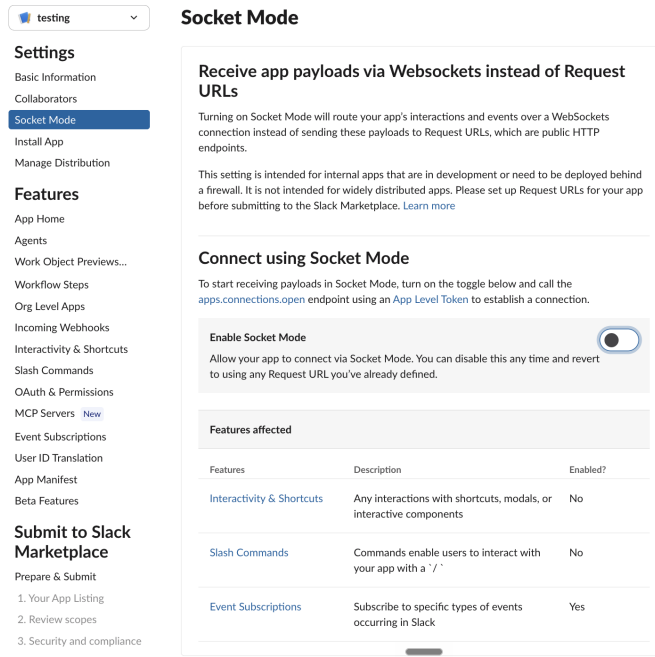
Signing Secret (from Slack's Basic Information page, Step 5) → paste into n8n's **Signature Secret** field.



n8n's Slack credential screen. **Access Token** holds the xoxb- Bot Token, **Signature Secret** holds Slack's Signing Secret. "Connection tested successfully" confirms the token is valid.

7. Important — Turn OFF Socket Mode

If **Socket Mode** is turned on in your Slack app, Slack routes events over a WebSocket connection instead of sending them to your Request URL (the n8n webhook). This means your Slack Trigger node in n8n will **never receive events**, even if your Request URL shows as "Verified." Go to **Socket Mode** in the sidebar and make sure **Enable Socket Mode** is switched **OFF**, unless you specifically know you need it.



Socket Mode page. The toggle shown here is **ON** — this is the single most common reason a correctly-scoped, correctly-verified Slack app still doesn't deliver messages into n8n. Switch it off to let the Request URL (webhook) path work.

8. Add a Slack Node to Send Messages

Elsewhere in your n8n workflow, add a regular **Slack** node (not the Trigger). Select the same credential, set **Resource** → Message, **Operation** → Send, choose the target channel, and map in your message text.

9. Invite the Bot and Test

1. In Slack, open the target channel and type `/invite @YourAppName` so the bot can post and read there.
2. Send a test message in that channel — check that the Slack Trigger node in n8n fires and shows the message data.
3. Manually execute your workflow's Send-message branch and confirm the message appears in Slack.

Troubleshooting Quick Reference

Symptom	Likely Cause	Fix
---------	--------------	-----

Channel dropdown is empty / red warning triangle	Missing channels:read or groups:read scope, or app not reinstalled after adding scopes	Add the missing :read scope, then reinstall the app (Step 5)
Request URL won't verify	n8n workflow not active, or wrong URL pasted	Activate the workflow / use the Test URL while testing
Everything verified, but no events arrive	Socket Mode is enabled	Turn OFF Socket Mode (Step 7)
Bot can't see a channel	Bot not invited to that channel	Run /invite @YourAppName in that channel